Alex Castillo González

Software Engineer · Python / TypeScript / Ruby

Santo Domingo, Dominican Republic — UTC−4 · Independent contractor (Spanish passport) · Spanish (native), English (professional) alex.castillog33@gmail.com · github.com/pcbeingused333 · linkedin.com/in/alexcastillogonzalez · portfolio-alexgonzalez33.vercel.app

SUMMARY

Software engineer working across Python, TypeScript and Ruby, with **209 merged pull requests** across 75 open-source projects this year (84% of those decided were merged): twelve in **Haystack** and its integrations (deepset's RAG and agent framework), 30 in **LangChain.rb**, 13 in the Rails app **Sure** (personal finance, 10k★), nine in **pipenv**, **LiteLLM** and **redis-py**, and fixes in Ruby tooling (`rubocop-rspec`, `courrier`, `perron`) and Python libraries (`pydantic-ai`, `opik`, `phoenix`, `pdm`) — concurrency, serialization and error-handling defects. I build and deploy end to end: Python services with their own test suites, Next.js front ends, AWS infrastructure in Terraform with CI/CD on GitHub Actions. Earlier, fullstack Rails at TECNOBIT and teaching Ruby and SQL at Le Wagon. Before moving to the Dominican Republic I founded and ran a business on my own in Toronto.

SKILLS

Backend — Python · pytest · Ruby on Rails · PostgreSQL · REST APIs · JSON/API integrations (Jira API)

Web — TypeScript · Next.js · React · Tailwind · JavaScript · HTML/CSS

Infra & tooling — **AWS** (Lambda container images, DynamoDB, ECR, IAM least-privilege, CloudWatch) · **Terraform** · **CI/CD with GitHub Actions, OIDC federation** · Docker · Vercel · Git (GitHub, GitLab) · AI-assisted development (Claude Code, Cursor)

AI / LLM — LLM APIs · LangChain · LangGraph · agents and tool use · **Model Context Protocol (MCP)** servers and clients · RAG with embeddings, FAISS and `pgvector` · LLM evaluation

EXPERIENCE

Churrería Calderón — Founder & owner-operator · Toronto, Canada

Sep 2025 – Jul 2026 *(business closed July 2026)*

- Moved my business from Spain to Toronto on my own: leased premises in Kensington Market, imported the machinery, registered the business and obtained every permit to operate legally. Opened in December 2025 and ran it with no hires — production, service, purchasing, cash and compliance — until it closed in July 2026.

Family churrería business — Manager · Catalonia, Spain

2023 – Aug 2025 · mainly summer seasons *(and in the same business before that)*

- Managed the family business, mainly in the summer seasons — production, service, purchasing and cash — and took it from a mobile trailer to fixed premises, rebuilding the operation around a different site, hours and customer base.

Le Wagon — Part-time Programming Teacher · Remote (France)

Oct 2022

- Taught the new part-time flex cohort of the fullstack bootcamp — Ruby, OOP, SQL and PostgreSQL, HTML/CSS/JavaScript, Rails — through exercises and live debugging.

TECNOBIT (Grupo Oesía) — Fullstack Developer · Valdepeñas, Spain

Aug 2022 – Nov 2022 · On-site

- Shipped features into a long-running internal application maintained by a team of five, across a codebase distributed over both GitHub and GitLab.
- **Backend:** Ruby routines pulling data from JSON files and the **Jira API**, persisting it to PostgreSQL with multi-stage validation gating document generation and downloads. **Frontend:** form-driven pages and PostgreSQL-backed views with role-dependent data, surfacing subprocess state so users could follow a download to completion.

SELECTED PROJECTS

Business Ops Agent — MCP server + agent, with trajectory evaluation

Live demo · Code

A **Model Context Protocol server** exposing a business's operations (catalog, booking capacity, stock, quoting, orders) as tools any MCP client can call — Claude Desktop, Cursor, or the LangGraph agent bundled with it. The agent carries no business rules: tools are discovered at runtime, so adding one requires no agent change.

- Built an evaluation harness that scores **tool trajectories**, not just answers: which tools were called, in what order, with which arguments, and whether every figure in the reply traces back to a tool result — deterministic, no judge model, so it holds even when a fabricated number happens to be correct. It caught the agent answering with zero tool calls, and caught it **intermittently**, which single-run testing misses. 11/12, mean 0.98.

- **Deployed to AWS** as a remote MCP server — Lambda container behind a Function URL, DynamoDB single-table store, least-privilege IAM, all in Terraform. Storage sits behind an interface, so the same server runs on SQLite locally and DynamoDB in production.
- Python, MCP 2.0, LangGraph, Groq, AWS (Lambda, DynamoDB, ECR, IAM), Terraform, Docker, pytest (161 tests). CI/CD on every push via GitHub Actions authenticating with **OIDC** rather than a stored key, scoped to one branch and unable to apply infrastructure.

Ask the GDPR — retrieval over regulation, with citations that can be checked

Live demo · Code

LangGraph agent over the full text of the GDPR. Every answer names the provision behind it — `Art. 33(1)`, not a page number — and the system declines when the source does not cover the question. Two modes behind one flag: an in-memory FAISS demo on a free 1 GB container, and a pgvector-backed production path.

- **Made the citation structural rather than incidental.** A regulation is cited by article and paragraph; the page a provision lands on is an artefact of typesetting. So the corpus is not a PDF: a builder parses the Official Journal text from EUR-Lex into **414 provisions** carrying article, paragraph and chapter as metadata, and the chunk size was chosen so **97% survive as exactly one chunk** — a chunk straddling Art. 33(1) and 33(2) cites the wrong paragraph.
- **Built an eval for the answers that should never be given.** In legal text a retrieval miss announces itself; an invention is fluent and indistinguishable from a correct answer. The harness scores refusal against questions the Regulation does not answer but every model has read about — adequacy decisions, Schrems II, a CCPA penalty — plus a deterministic check for citations that appear in the answer and never in the passages.
- **Re-ran every measurement when the corpus changed, and one result reversed.** Embedding the article heading hurt retrieval under the old embedding model and helped under the new one; carrying the first conclusion forward would have shipped the worse setting on the strength of real evidence. The model swap was worth 8/20 → 13/20 at rank 1 for 42 MB, against a hard 1 GB ceiling.
- Python, LangChain, LangGraph, Groq, FAISS, pgvector, Streamlit, Docker, GitHub Actions, pytest (72 tests). CI runs headlessly on every push, including a boot test of the app — the host redeploys straight from `main`, so the suite is the only gate before the demo.

ragcite — the evaluation layer of the two projects above, as a library

Code

Retrieval metrics, citation grounding, abstention scoring and tool-trajectory scoring, extracted from the two harnesses above and generalized. No LangChain, vector store or LLM client is imported: you bring the retriever and the judge, `ragcite` scores what they return.

- **It reproduces the numbers of the project it came from, and asserts it.** The dogfood example scores one FAISS index twice — once with `ragcite`, once with the original project's own independent metrics code — and exits non-zero if they disagree. They match exactly (hit@1 13/25, recall@k 17/25, MRR 0.59).
- Zero runtime dependencies, MIT, 155 tests, CI on Python 3.9–3.12. `--min-hit-at-1` turns a run into a CI gate that exits 2 on a regression.

OPEN SOURCE

209 merged pull requests across 75 open-source projects since July 2026, 84% of those decided; median 11 hours to merge. Mostly the layer between applications and LLM providers, each with a regression test that fails without the fix:

- **LiteLLM** (#44795): streamed Vertex AI pass-through calls were priced without their location, so regional endpoints missed their 10% uplift; now the logger prices streamed and non-streamed calls the same way.
- **LLM Tornado** (#205): Anthropic streaming re-yielded every tool call at each block end, so three parallel tools ran six times and the follow-up was rejected; calls are now yielded once with the final usage. 13 merged there.
- **Haystack** (#12662): `run_async` had drifted from `run` — no concurrency limit, a lock shared across event loops. 12 merged in deepset's framework (write-up).
- **LangChain.rb** (30 merged): `tool_choice` shapes, forced-tool loops, streamed tool calls and thinking blocks across six provider adapters, most merged the same day.
- **Model retirements** (AgentScope-Java #3670): replacing a retiring default model, plus the context-window entry without which the swap would have resolved the window to 0.

EDUCATION

Le Wagon — Fullstack Web Development bootcamp, 2022. Self-directed study since.